In [ ]:
```
# Add --break-system-packages at the end of the pip install
# for local installs on OS's like Ubuntu
!git clone https://github.com/ucsd-cse150b-f25/notebooks  > /dev/null 2>&1
!mv notebooks/* ./
# After first run, comment the line and rerun
```

In [ ]:
```
# Run once and restart your session.
# After restart comment the line and rerun
!pip install -r requirements.txt > /dev/null 2>&1
```

In [ ]:
```
# After pip install, run this once, restart your session
# After restart comment the line and rerun
!playwright install chromium > /dev/null 2>&1
```

Making Complex Decisions

This Jupyter notebook acts as supporting material for topics covered in **Chapter 17 Making Complex Decisions** of the book* Artificial Intelligence: A Modern Approach*. We make use of the implementations in mdp.py module. This notebook also includes a brief summary of the main topics as a review. Let us import everything from the mdp module to get started.

In [83]:
```
from mdp import *
from notebook import psource, pseudocode, plot_pomdp_utility
from IPython.display import display, Image
import imgkit
%matplotlib inline
```

CONTENTS

OVERVIEW

Before we start playing with the actual implementations let us review a couple of things about MDPs.

- A stochastic process has the **Markov property** if the conditional probability distribution of future states of the process (conditional on both past and present states) depends only upon the present state, not on the sequence of events that preceded it.

  -- Source: Wikipedia

Often it is possible to model many different phenomena as a Markov process by being flexible with our definition of state.

- MDPs help us deal with fully-observable and non-deterministic/stochastic environments. For dealing with partially-observable and stochastic cases we make use of generalization of MDPs named POMDPs (partially observable Markov decision process).

Our overall goal to solve a MDP is to come up with a policy which guides us to select the best action in each state so as to maximize the expected sum of future rewards.

MDP

To begin with let us look at the implementation of MDP class defined in mdp.py The docstring tells us what all is required to define a MDP namely - set of states, actions, initial state, transition model, and a reward function. Each of these are implemented as methods. Do not close the popup so that you can follow along the description of code below.

In [ ]:
```
psource(MDP)
```

The _ ***init*** _ method takes in the following parameters:

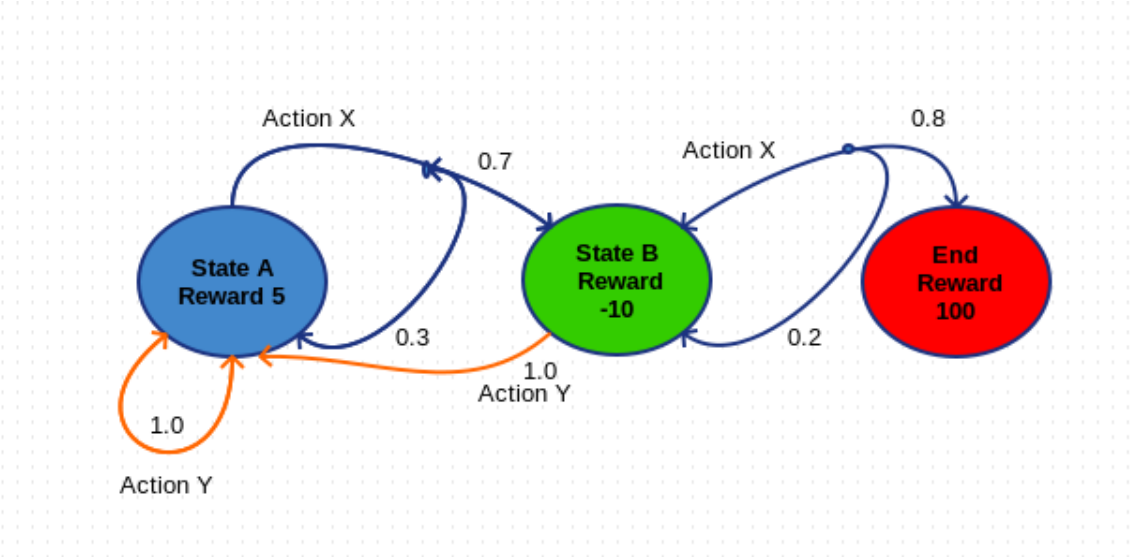
- init: the initial state.
- actlist: List of actions possible in each state.
- terminals: List of terminal states where only possible action is exit
- gamma: Discounting factor. This makes sure that delayed rewards have less value compared to immediate ones.

R method returns the reward for each state by using the self.reward dict.

T method is not implemented and is somewhat different from the text. Here we return (probability, s') pairs where s' belongs to list of possible state by taking action a in state s.

actions method returns list of actions possible in each state. By default it returns all actions for states other than terminal states.

Now let us implement the simple MDP in the image below. States A, B have actions X, Y available in them. Their probabilities are shown just above the arrows. We start with using MDP as base class for our CustomMDP. Obviously we need to make a few changes to suit our case. We make use of a transition matrix as our transitions are not very simple.



In [ ]:
```
# Transition Matrix as nested dict. State -> Actions in state -> List of (Pr
t = {
    "A": {
            "X": [(0.3, "A"), (0.7, "B")],
            "Y": [(1.0, "A")]
        },
    "B": {
            "X": {(0.8, "End"), (0.2, "B")},
            "Y": {(1.0, "A")}
        },
    "End": {}
}

init = "A"

terminals = ["End"]

rewards = {
    "A": 5,
    "B": -10,
    "End": 100
}
```

In [ ]:
```
class CustomMDP(MDP):
    def __init__(self, init, terminals, transition_matrix, reward = None, ga
        # All possible actions.
        actlist = []
```

```
        for state in transition_matrix.keys():
            actlist.extend(transition_matrix[state])
        actlist = list(set(actlist))
        MDP.__init__(self, init, actlist, terminals, transition_matrix, rewa

    def T(self, state, action):
        if action is None:
            return [(0.0, state)]
        else:
            return self.t[state][action]
```

Finally we instantize the class with the parameters for our MDP in the picture.

In [ ]:
```
our_mdp = CustomMDP(init, terminals, t, rewards, gamma=.9)
```

With this we have successfully represented our MDP. Later we will look at ways to solve this MDP.

GRID MDP

Now we look at a concrete implementation that makes use of the MDP as base class. The GridMDP class in the mdp module is used to represent a grid world MDP like the one shown in in **Fig 17.1** of the AIMA Book. We assume for now that the environment is *fully observable*, so that the agent always knows where it is. The code should be easy to understand if you have gone through the CustomMDP example.

In [ ]:
```
psource(GridMDP)
```

The _ ***init*** _ method takes **grid** as an extra parameter compared to the MDP class. The grid is a nested list of rewards in states.

go method returns the state by going in particular direction by using vector_add.

T method is not implemented and is somewhat different from the text. Here we return (probability, s') pairs where s' belongs to list of possible state by taking action a in state s.

actions method returns list of actions possible in each state. By default it returns all actions for states other than terminal states.

to_arrows are used for representing the policy in a grid like format.

We can create a GridMDP like the one in **Fig 17.1** as follows:

```
GridMDP([[-0.04, -0.04, -0.04, +1],
         [-0.04, None,  -0.04, -1],
         [-0.04, -0.04, -0.04, -0.04]],
```

```
                    terminals=[(3, 2), (3, 1)])
```

In fact the **sequential_decision_environment** in mdp module has been instantized using the exact same code.

In [ ]:
```
sequential_decision_environment
```

VALUE ITERATION

Now that we have looked how to represent MDPs. Let's aim at solving them. Our ultimate goal is to obtain an optimal policy. We start with looking at Value Iteration and a visualisation that should help us understanding it better.

We start by calculating Value/Utility for each of the states. The Value of each state is the expected sum of discounted future rewards given we start in that state and follow a particular policy π. The value or the utility of a state is given by

$$U(s) = R(s) + \gamma \max_{a \epsilon A(s)} \sum_{s'} P(s' \mid s, a) U(s')$$

This is called the Bellman equation. The algorithm Value Iteration (**Fig. 17.4** in the book) relies on finding solutions of this Equation. The intuition Value Iteration works is because values propagate through the state space by means of local updates. This point will we more clear after we encounter the visualisation. For more information you can refer to **Section 17.2** of the book.

In [ ]:
```
psource(value_iteration)
```

It takes as inputs two parameters, an MDP to solve and epsilon, the maximum error allowed in the utility of any state. It returns a dictionary containing utilities where the keys are the states and values represent utilities.
Value Iteration starts with arbitrary initial values for the utilities, calculates the right side of the Bellman equation and plugs it into the left hand side, thereby updating the utility of each state from the utilities of its neighbors. This is repeated until equilibrium is reached. It works on the principle of *Dynamic Programming* - using precomputed information to simplify the subsequent computation. If $U_i(s)$ is the utility value for state s at the i th iteration, the iteration step, called Bellman update, looks like this:

$$U_{i+1}(s) \leftarrow R(s) + \gamma \max_{a \epsilon A(s)} \sum_{s'} P(s' \mid s, a) U_i(s')$$

As you might have noticed, `value_iteration` has an infinite loop. How do we decide when to stop iterating? The concept of *contraction* successfully explains the convergence of value iteration. Refer to **Section 17.2.3** of the book for a detailed explanation. In the algorithm, we calculate a value $delta$ that measures the difference in the utilities of the current time step and the previous time step.

$$\delta = \max\left(\delta, \left| U_{i+1}(s) - U_i(s) \right|\right)$$

This value of delta decreases as the values of U_i converge. We terminate the algorithm if the δ value is less than a threshold value determined by the hyperparameter *epsilon*.

$$\delta < \epsilon\frac{(1-\gamma)}{\gamma}$$

To summarize, the Bellman update is a *contraction* by a factor of $gamma$ on the space of utility vectors. Hence, from the properties of contractions in general, it follows that `value_iteration` always converges to a unique solution of the Bellman equations whenever $gamma$ is less than 1. We then terminate the algorithm when a reasonable approximation is achieved. In practice, it often occurs that the policy pi becomes optimal long before the utility function converges. For the given 4 x 3 environment with $gamma = 0.9$, the policy pi is optimal when $i = 4$ (at the 4th iteration), even though the maximum error in the utility function is stil 0.46. This can be clarified from **figure 17.6** in the book. Hence, to increase computational efficiency, we often use another method to solve MDPs called Policy Iteration which we will see in the later part of this notebook.
For now, let us solve the **sequential_decision_environment** GridMDP using `value_iteration`.

In [ ]:
```
value_iteration(sequential_decision_environment)
```

The pseudocode for the algorithm:

In [ ]:
```
pseudocode("Value-Iteration")
```

AIMA3e

function VALUE-ITERATION(*mdp*, *ε*) **returns** a utility function
  inputs: *mdp*, an MDP with states *S*, actions *A*(*s*), transition model *P*(*s*′ | *s*, *a*),
    rewards *R*(*s*), discount *γ*
    ε, the maximum error allowed in the utility of any state
  local variables: *U*, *U*′, vectors of utilities for states in *S*, initially zero
    δ, the maximum change in the utility of any state in an iteration

repeat
  $U \leftarrow U'$; $\delta \leftarrow 0$
  for each state s in S **do**
    $U'[s] \leftarrow R(s) + \gamma \max a \in A(s) \Sigma P(s' \mid s, a) U[s']$
    if $\mid U'[s] - U[s] \mid > \delta$ **then** $\delta \leftarrow \mid U'[s] - U[s] \mid$
until $\delta < \varepsilon(1 - \gamma)/\gamma$
return U

Figure ?? The value iteration algorithm for calculating utilities of states. The termination condition is from Equation (**??**).

VALUE ITERATION VISUALIZATION

To illustrate that values propagate out of states let us create a simple visualisation. We will be using a modified version of the value_iteration function which will store U over time. We will also remove the parameter epsilon and instead add the number of iterations we want.

In [ ]:
```
def value_iteration_instru(mdp, iterations=20):
    U_over_time = []
    U1 = {s: 0 for s in mdp.states}
    R, T, gamma = mdp.R, mdp.T, mdp.gamma
    for _ in range(iterations):
        U = U1.copy()
        for s in mdp.states:
            U1[s] = R(s) + gamma * max([sum([p * U[s1] for (p, s1) in T(s, a
                                        for a in mdp.actions(s)])
        U_over_time.append(U)
    return U_over_time
```

Next, we define a function to create the visualisation from the utilities returned by **value_iteration_instru**. The reader need not concern himself with the code that immediately follows as it is the usage of Matplotib with IPython Widgets. If you are interested in reading more about these visit ipywidgets.readthedocs.io

In [ ]:
```
columns = 4
rows = 3
U_over_time = value_iteration_instru(sequential_decision_environment)
```

In [ ]:
```
%matplotlib inline
from notebook import make_plot_grid_step_function

plot_grid_step = make_plot_grid_step_function(columns, rows, U_over_time)
```

In [ ]:
```
import ipywidgets as widgets
from IPython.display import display
from notebook import make_visualize
```

```
iteration_slider = widgets.IntSlider(min=1, max=15, step=1, value=0)
w=widgets.interactive(plot_grid_step,iteration=iteration_slider)
display(w)

visualize_callback = make_visualize(iteration_slider)

visualize_button = widgets.ToggleButton(description = "Visualize", value = F
time_select = widgets.ToggleButtons(description='Extra Delay:',options=['0',
a = widgets.interactive(visualize_callback, Visualize = visualize_button, ti
display(a)
```

Move the slider above to observe how the utility changes across iterations. It is also possible to move the slider using arrow keys or to jump to the value by directly editing the number with a double click. The **Visualize Button** will automatically animate the slider for you. The **Extra Delay Box** allows you to set time delay in seconds upto one second for each time step. There is also an interactive editor for grid-world problems `grid_mdp.py` in the gui folder for you to play around with.

POLICY ITERATION

We have already seen that value iteration converges to the optimal policy long before it accurately estimates the utility function. If one action is clearly better than all the others, then the exact magnitude of the utilities in the states involved need not be precise. The policy iteration algorithm works on this insight. The algorithm executes two fundamental steps:

- **Policy evaluation**: Given a policy π_i, calculate $U_i = U(\pi_i)$, the utility of each state if π_i were to be executed.
- **Policy improvement**: Calculate a new policy π_{i+1} using one-step look-ahead based on the utility values calculated.

The algorithm terminates when the policy improvement step yields no change in the utilities. Refer to **Figure 17.6** in the book to see how this is an improvement over value iteration. We now have a simplified version of the Bellman equation

$$U_i(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi_i(s)) U_i(s')$$

An important observation in this equation is that this equation doesn't have the `max` operator, which makes it linear. For n states, we have n linear equations with n unknowns, which can be solved exactly in time $O(n^3)$. For more implementational details, have a look at **Section 17.3**. Let us now look at how the expected utility is found and how `policy_iteration` is implemented.

In [ ]:
```
psource(expected_utility)
```

In [ ]:
```
psource(policy_iteration)
```

Fortunately, it is not necessary to do *exact* policy evaluation. The utilities can instead be reasonably approximated by performing some number of simplified value iteration steps. The simplified Bellman update equation for the process is

$$U_{i+1}(s) \leftarrow R(s) + \gamma \sum_{s'} P(s' \mid s, \pi_i(s)) U_i(s')$$

and this is repeated *k* times to produce the next utility estimate. This is called *modified policy iteration*.

In [ ]:
```
psource(policy_evaluation)
```

Let us now solve `sequential_decision_environment` using `policy_iteration`.

In [ ]:
```
policy_iteration(sequential_decision_environment)
```

In [ ]:
```
pseudocode('Policy-Iteration')
```

AIMA3e

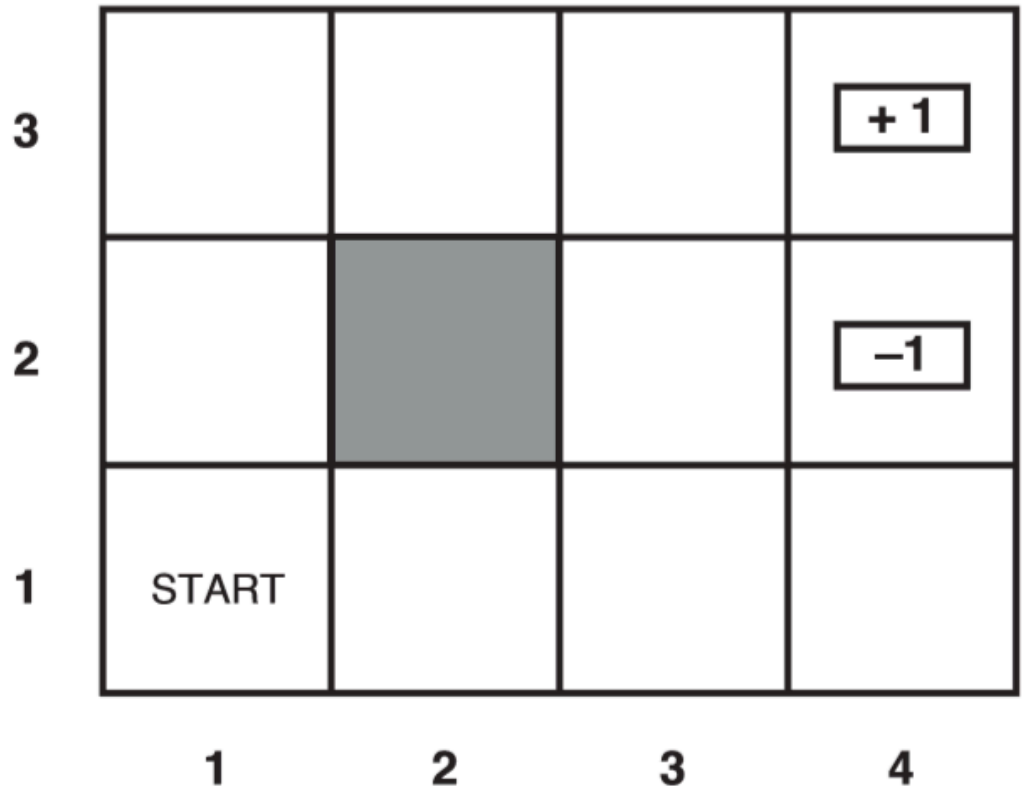
function POLICY-ITERATION(*mdp*) **returns** a policy
  inputs: *mdp*, an MDP with states *S*, actions *A*(*s*), transition model *P*(*s*′ | *s*, *a*)
  local variables: *U*, a vector of utilities for states in *S*, initially zero
    π, a policy vector indexed by state, initially random

  repeat
    U ← POLICY-EVALUATION(*π*, *U*, *mdp*)
    unchanged? ← true
    for each state *s* **in** *S* **do**
      if max*a* ∈ *A*(*s*) Σ*s*′ *P*(*s*′ | *s*, *a*) *U*[*s*′] > Σ*s*′ *P*(*s*′ | *s*, *π*[*s*]) *U*[*s*′] **then do**
        π[*s*] ← argmax*a* ∈ *A*(*s*) Σ*s*′ *P*(*s*′ | *s*, *a*) *U*[*s*′]
        unchanged? ← false
  until *unchanged?*
  return *π*

Figure ?? The policy iteration algorithm for calculating an optimal policy.

Sequential Decision Problems

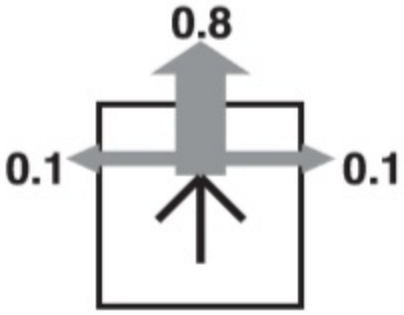
Now that we have the tools required to solve MDPs, let us see how Sequential Decision Problems can be solved step by step and how a few built-in tools in the GridMDP class help us better analyse the problem at hand. As always, we will work with the grid world from **Figure 17.1** from the book.



This is the environment for our agent. We assume for now that the environment is *fully observable*, so that the agent always knows where it is. We also assume that the transitions are **Markovian**, that is, the probability of reaching state s' from state s depends only on s and not on the history of earlier states. Almost all stochastic decision problems can be reframed as a Markov Decision Process just by tweaking the definition of a *state* for that particular problem.
However, the actions of our agent in this environment are unreliable. In other words, the motion of our agent is stochastic.

More specifically, the agent may -

- move correctly in the intended direction with a probability of *0.8*,
- move 90° to the right of the intended direction with a probability 0.1
- move 90° to the left of the intended direction with a probability 0.1



The agent stays put if it bumps into a wall.

These properties of the agent are called the transition properties and are hardcoded into the GridMDP class as you can see below.

In [ ]:
```
psource(GridMDP.T)
```

To completely define our task environment, we need to specify the utility function for the agent. This is the function that gives the agent a rough estimate of how good being in a particular state is, or how much *reward* an agent receives by being in that state. The agent then tries to maximize the reward it gets. As the decision problem is sequential, the utility function will depend on a sequence of states rather than on a single state. For now, we simply stipulate that in each state s, the agent receives a finite reward $R(s)$.

For any given state, the actions the agent can take are encoded as given below:

- Move Up: (0, 1)
- Move Down: (0, -1)
- Move Left: (-1, 0)
- Move Right: (1, 0)
- Do nothing: `None`

We now wonder what a valid solution to the problem might look like. We cannot have fixed action sequences as the environment is stochastic and we can eventually end up in an undesirable state. Therefore, a solution must specify what the agent shoulddo for *any* state the agent might reach.
Such a solution is known as a **policy** and is usually denoted by π.
The **optimal policy** is the policy that yields the highest expected utility an is usually denoted by π^*.
The `GridMDP` class has a useful method `to_arrows` that outputs a grid showing the direction the agent should move, given a policy. We will use this later to better understand the properties of the environment.

In [ ]:
```
psource(GridMDP.to_arrows)
```

This method directly encodes the actions that the agent can take (described above) to characters representing arrows and shows it in a grid format for human visalization purposes. It converts the received policy from a `dictionary` to a grid using the `to_grid` method.

In [ ]:
```
psource(GridMDP.to_grid)
```

Now that we have all the tools required and a good understanding of the agent and the environment, we consider some cases and see how the agent should behave for each case.

Case 1

R(s) = -0.04 in all states except terminal states

In [ ]:
```
# Note that this environment is also initialized in mdp.py by default
sequential_decision_environment = GridMDP([[-0.04, -0.04, -0.04, +1],
                                           [-0.04, None, -0.04, -1],
                                           [-0.04, -0.04, -0.04, -0.04]],
                                          terminals=[(3, 2), (3, 1)])
```

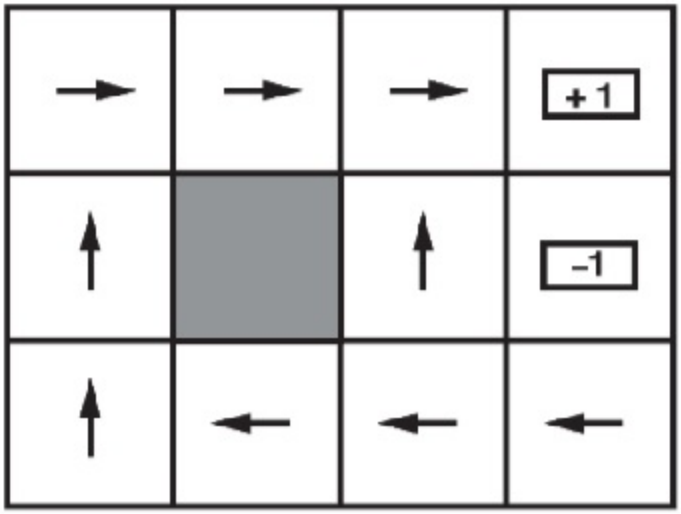
We will use the `best_policy` function to find the best policy for this environment. But, as you can see, `best_policy` requires a utility function as well. We already know that the utility function can be found by `value_iteration`. Hence, our best policy is:

In [ ]:
```
pi = best_policy(sequential_decision_environment, value_iteration(sequential
```

We can now use the `to_arrows` method to see how our agent should pick its actions in the environment.

In [ ]:
```
from utils import print_table
print_table(sequential_decision_environment.to_arrows(pi))
```

This is exactly the output we expected



Notice that, because the cost of taking a step is fairly small compared with the penalty for ending up in `(4, 2)` by accident, the optimal policy is conservative. In state `(3, 1)` it recommends taking the long way round, rather than taking the shorter way and risking getting a large negative reward of -1 in `(4, 2)`.

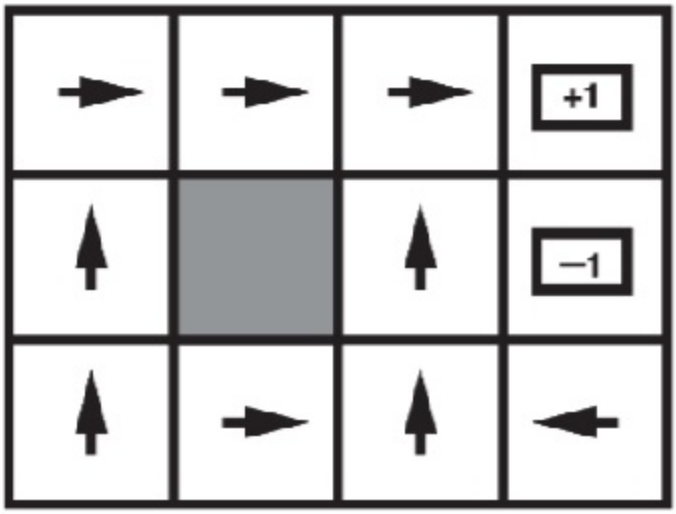
Case 2

R(s) = -0.4 in all states except in terminal states

In [ ]:
```
sequential_decision_environment = GridMDP([[-0.4, -0.4, -0.4, +1],
                                           [-0.4, None, -0.4, -1],
                                           [-0.4, -0.4, -0.4, -0.4]],
                                          terminals=[(3, 2), (3, 1)])
```

In [ ]:
```
pi = best_policy(sequential_decision_environment, value_iteration(sequential
from utils import print_table
print_table(sequential_decision_environment.to_arrows(pi))
```

This is exactly the output we expected



As the reward for each state is now more negative, life is certainly more unpleasant. The agent takes the shortest route to the +1 state and is willing to risk falling into the -1 state by accident.

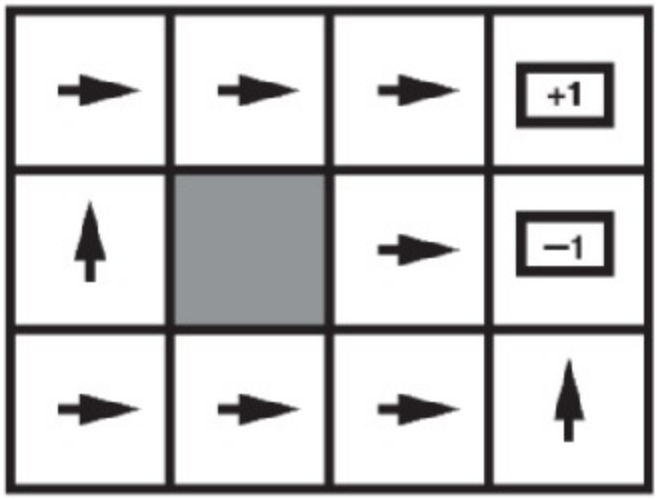
Case 3

R(s) = -4 in all states except terminal states

In [ ]:
```
sequential_decision_environment = GridMDP([[-4, -4, -4, +1],
                                           [-4, None, -4, -1],
                                           [-4, -4, -4, -4]],
                                          terminals=[(3, 2), (3, 1)])
```

In [ ]:
```
pi = best_policy(sequential_decision_environment, value_iteration(sequential
from utils import print_table
print_table(sequential_decision_environment.to_arrows(pi))
```

This is exactly the output we expected



The living reward for each state is now lower than the least rewarding terminal. Life is so *painful* that the agent heads for the nearest exit as even the worst exit is less painful than any living state.

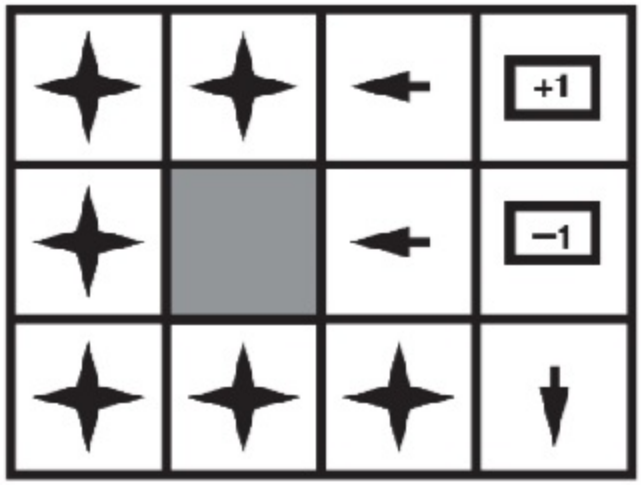
Case 4

R(s) = 4 in all states except terminal states

In [ ]:
```
sequential_decision_environment = GridMDP([[4, 4, 4, +1],
                                           [4, None, 4, -1],
                                           [4, 4, 4, 4]],
                                          terminals=[(3, 2), (3, 1)])
```

In [ ]:
```
pi = best_policy(sequential_decision_environment, value_iteration(sequential
from utils import print_table
print_table(sequential_decision_environment.to_arrows(pi))
```

In this case, the output we expect is



As life is positively enjoyable and the agent avoids *both* exits. Even though the output we get is not exactly what we want, it is definitely not wrong. The scenario here requires the agent to anything but reach a terminal state, as this is the only way the agent can maximize its reward (total reward tends to infinity), and the program does just that.
Currently, the GridMDP class doesn't support an explicit marker for a "do whatever you like" action or a "don't care" condition. You can however, extend the class to do so.
For in-depth knowledge about sequential decision problems, refer **Section 17.1** in the AIMA book.

POMDP

Partially Observable Markov Decision Problems

In retrospect, a Markov decision process or MDP is defined as:

- a sequential decision problem for a fully observable, stochastic environment with a Markovian transition model and additive rewards.

An MDP consists of a set of states (with an initial state s_0); a set $A(s)$ of actions in each state; a transition model $P(s'|s,a)$; and a reward function $R(s)$.

The MDP seeks to make sequential decisions to occupy states so as to maximise some combination of the reward function $R(s)$.

The characteristic problem of the MDP is hence to identify the optimal policy function $\pi^*(s)$ that provides the *utility-maximising* action a to be taken when the current state is s.

Belief vector

Note: The book refers to the *belief vector* as the *belief state*. We use the latter terminology here to retain our ability to refer to the belief vector as a *probability distribution over states*.

The solution of an MDP is subject to certain properties of the problem which are assumed and justified in [Section 17.1]. One critical assumption is that the agent is **fully aware of its current state at all times**.

A tedious (but rewarding, as we will see) way of expressing this is in terms of the **belief vector** b of the agent. The belief vector is a function mapping states to probabilities or certainties of being in those states.

Consider an agent that is fully aware that it is in state s_i in the statespace $(s_1, s_2, \ldots s_n)$ at the current time.

Its belief vector is the vector $(b(s_1), b(s_2), \ldots b(s_n))$ given by the function $b(s)$:

$$\begin{aligned} b(s) &= 0 \quad \text{if } s \neq s_i \\ &= 1 \quad \text{if } s = s_i \end{aligned}$$

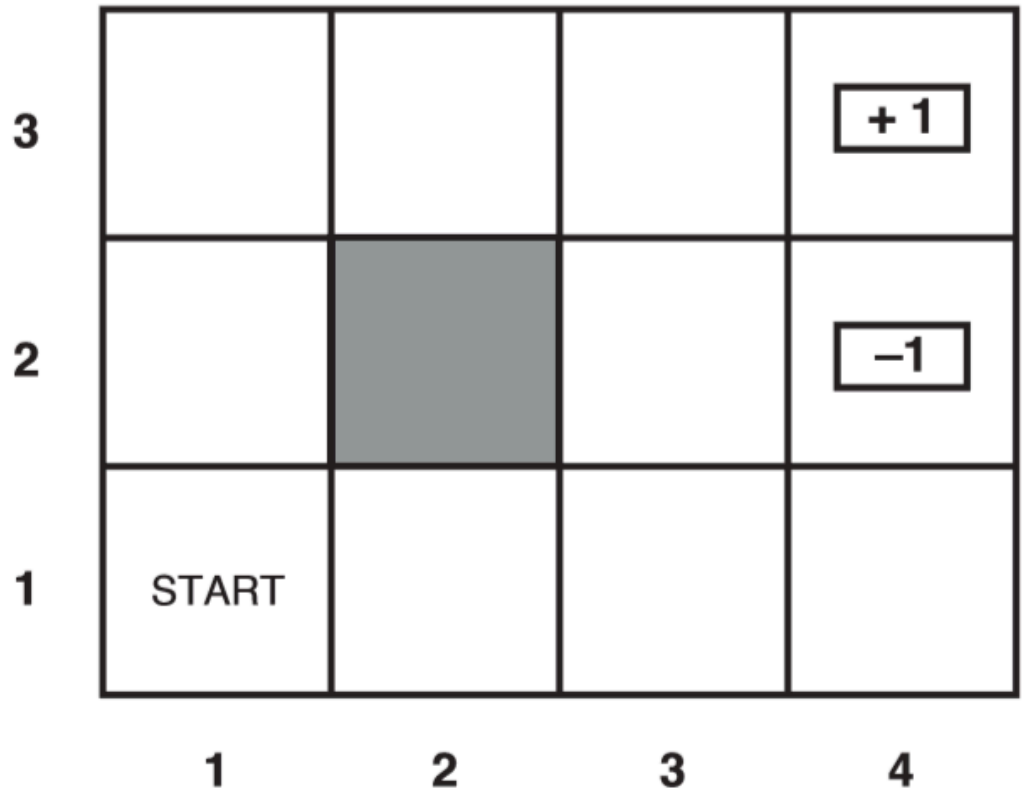
Note that $b(s)$ is a probability distribution that necessarily sums to 1 over all s.

POMDPs - a conceptual outline

The POMDP really has only two modifications to the **problem formulation** compared to the MDP.

- **Belief state** - In the real world, the current state of an agent is often not known with complete certainty. This makes the concept of a belief vector extremely relevant. It allows the agent to represent different degrees of certainty with which it *believes* it is in each state.
- **Evidence percepts** - In the real world, agents often have certain kinds of evidence, collected from sensors. They can use the probability distribution of observed evidence, conditional on state, to consolidate their information. This is a known distribution $P(e \mid s)$ - e being an evidence, and s being the state it is conditional on.

Consider the world we used for the MDP.



Using the belief vector

An agent beginning at $(1, 1)$ may not be certain that it is indeed in $(1, 1)$. Consider a belief vector b such that:

$$\begin{aligned} b((1,1)) &= 0.8 \\ b((2,1)) &= 0.1 \\ b((1,2)) &= 0.1 \\ b(s) &= 0 \qquad \forall \text{ other } s \end{aligned}$$

By horizontally catenating each row, we can represent this as an 11-dimensional vector (omitting $(2, 2)$).

Thus, taking $s_1 = (1, 1)$, $s_2 = (1, 2)$, ... $s_{11} = (4, 3)$, we have b:

$b = (0.8, 0.1, 0, 0, 0.1, 0, 0, 0, 0, 0, 0)$

This fully represents the certainty to which the agent is aware of its state.

Using evidence

The evidence observed here could be the number of adjacent 'walls' or 'dead ends' observed by the agent. We assume that the agent cannot 'orient' the walls - only count them.

In this case, e can take only two values, 1 and 2. This gives $P(e \mid s)$ as:

$$P(e = 2 \mid s) = \frac{1}{7} \quad \forall \quad s \in \{s_1, s_2, s_4, s_5, s_8, s_9, s_{11}\}$$
$$P(e = 1 \mid s) = \frac{1}{4} \quad \forall \quad s \in \{s_3, s_6, s_7, s_{10}\}$$
$$P(e \mid s) = 0 \quad \forall \quad \text{other } s, e$$

Note that the implications of the evidence on the state must be known **a priori** to the agent. Ways of reliably learning this distribution from percepts are beyond the scope of this notebook.

POMDPs - a rigorous outline

A POMDP is thus a sequential decision problem for for a *partially* observable, stochastic environment with a Markovian transition model, a known 'sensor model' for inferring state from observation, and additive rewards.

Practically, a POMDP has the following, which an MDP also has:

- a set of states, each denoted by s
- a set of actions available in each state, $A(s)$
- a reward accrued on attaining some state, $R(s)$
- a transition probability $P(s' \mid s, a)$ of action a changing the state from s to s'

And the following, which an MDP does not:

- a sensor model $P(e \mid s)$ on evidence conditional on states

Additionally, the POMDP is now uncertain of its current state hence has:

- a belief vector b representing the certainty of being in each state (as a probability distribution)

New uncertainties

It is useful to intuitively appreciate the new uncertainties that have arisen in the agent's awareness of its own state.

- At any point, the agent has belief vector b, the distribution of its believed likelihood of being in each state s.
- For each of these states s that the agent may **actually** be in, it has some set of actions given by $A(s)$.
- Each of these actions may transport it to some other state s', assuming an initial state s, with probability $P(s' \mid s, a)$
- Once the action is performed, the agent receives a percept e. $P(e \mid s)$ now tells it the chances of having perceived e for each state s. The agent must

use this information to update its new belief state appropriately.

Evolution of the belief vector - the `FORWARD` function

The new belief vector $b'(s')$ after an action a on the belief vector $b(s)$ and the noting of evidence e is:

$$b'(s') = \alpha P(e \mid s') \sum_{s} P(s' \mid s, a) b(s)$$

where α is a normalising constant (to retain the interpretation of b as a probability distribution.

This equation is just counts the sum of likelihoods of going to a state s' from every possible state s, times the initial likelihood of being in each s. This is multiplied by the likelihood that the known evidence actually implies the new state s'.

This function is represented as `b' = FORWARD(b, a, e)`

Probability distribution of the evolving belief vector

The goal here is to find $P(b' \mid b, a)$ - the probability that action a transforms belief vector b into belief vector b'. The following steps illustrate this -

The probability of observing evidence e when action a is enacted on belief vector b can be distributed over each possible new state s' resulting from it:

$$\begin{aligned} P(e \mid b, a) &= \sum_{s'} P(e \mid b, a, s') P(s' \mid b, a) \\ &= \sum_{s'} P(e \mid s') P(s' \mid b, a) \\ &= \sum_{s'} P(e \mid s') \sum_{s} P(s' \mid s, a) b(s) \end{aligned}$$

The probability of getting belief vector b' from b by application of action a can thus be summed over all possible evidences e:

$$\begin{aligned} P(b' \mid b, a) &= \sum_{e} P(b' \mid b, a, e) P(e \mid b, a) \\ &= \sum_{e} P(b' \mid b, a, e) \sum_{s'} P(e \mid s') \sum_{s} P(s' \mid s, a) b(s) \end{aligned}$$

where $P(b' \mid b, a, e) = 1$ if $b' =$ `FORWARD(b, a, e)` and $= 0$ otherwise.

Given initial and final belief states b and b', the transition probabilities still depend on the action a and observed evidence e. Some belief states may be achievable by certain actions, but have non-zero probabilities for states

prohibited by the evidence e. Thus, the above condition thus ensures that only valid combinations of (b', b, a, e) are considered.

A modified rewardspace

For MDPs, the reward space was simple - one reward per available state. However, for a belief vector $b(s)$, the expected reward is now:

$$\rho(b) = \sum_s b(s)R(s)$$

Thus, as the belief vector can take infinite values of the distribution over states, so can the reward for each belief vector vary over a hyperplane in the belief space, or space of states (planes in an N-dimensional space are formed by a linear combination of the axes).

Now that we know the basics, let's have a look at the `POMDP` class.

In [ ]:
```
psource(POMDP)
```

The `POMDP` class includes all variables of the `MDP` class and additionally also stores the sensor model in `e_prob`.

`remove_dominated_plans`, `remove_dominated_plans_fast`, `generate_mapping` and `max_difference` are helper methods for `pomdp_value_iteration` which will be explained shortly.

To understand how we can model a partially observable MDP, let's take a simple example. Let's consider a simple two state world. The states are labelled 0 and 1, with the reward at state 0 being 0 and at state 1 being 1.
There are two actions:
`Stay`: stays put with probability 0.9 and `Go`: switches to the other state with probability 0.9.
For now, let's assume the discount factor `gamma` to be 1.
The sensor reports the correct state with probability 0.6.
This is a simple problem with a trivial solution. Obviously the agent should `Stay` when it thinks it is in state 1 and `Go` when it thinks it is in state 0.
The belief space can be viewed as one-dimensional because the two probabilities must sum to 1.

Let's model this POMDP using the `POMDP` class.

In [ ]:
```
# transition probability P(s'|s,a)
t_prob = [[[0.9, 0.1], [0.1, 0.9]], [[0.1, 0.9], [0.9, 0.1]]]
# evidence function P(e|s)
e_prob = [[[0.6, 0.4], [0.4, 0.6]], [[0.6, 0.4], [0.4, 0.6]]]
```

```
# reward function
rewards = [[0.0, 0.0], [1.0, 1.0]]
# discount factor
gamma = 0.95
# actions
actions = ('0', '1')
# states
states = ('0', '1')
```

In [ ]:
```
pomdp = POMDP(actions, t_prob, e_prob, rewards, states, gamma)
```

We have defined our `POMDP` object.

POMDP VALUE ITERATION

Defining a POMDP is useless unless we can find a way to solve it. As POMDPs can have infinitely many belief states, we cannot calculate one utility value for each state as we did in `value_iteration` for MDPs.
Instead of thinking about policies, we should think about conditional plans and how the expected utility of executing a fixed conditional plan varies with the initial belief state.
If we bound the depth of the conditional plans, then there are only finitely many such plans and the continuous space of belief states will generally be divided inte *regions*, each corresponding to a particular conditional plan that is optimal in that region. The utility function, being the maximum of a collection of hyperplanes, will be piecewise linear and convex.
For the one-step plans `Stay` and `Go`, the utility values are as follows

$$\alpha_{|Stay|}(0) = R(0) + \gamma(0.9R(0) + 0.1R(1)) = 0.1$$

$$\alpha_{|Stay|}(1) = R(1) + \gamma(0.9R(1) + 0.1R(0)) = 1.9$$

$$\alpha_{|Go|}(0) = R(0) + \gamma(0.9R(1) + 0.1R(0)) = 0.9$$

$$\alpha_{|Go|}(1) = R(1) + \gamma(0.9R(0) + 0.1R(1)) = 1.1$$

The utility function can be found by `pomdp_value_iteration`.
To summarize, it generates a set of all plans consisting of an action and, for each possible next percept, a plan in U with computed utility vectors. The dominated plans are then removed from this set and the process is repeated till the maximum difference between the utility functions of two consecutive iterations reaches a value less than a threshold value.

In [ ]:
```
pseudocode('POMDP-Value-Iteration')
```

Let's have a look at the `pomdp_value_iteration` function.

In [ ]:
```
psource(pomdp_value_iteration)
```

This function uses two aptly named helper methods from the `POMDP` class, `remove_dominated_plans` and `max_difference`.

Let's try solving a simple one-dimensional POMDP using value-iteration.
Consider the problem of a user listening to voicemails. At the end of each message, they can either *save* or *delete* a message. This forms the unobservable state *S = {save, delete}*. It is the task of the POMDP solver to guess which goal the user has.
The belief space has two elements, *b(s = save)* and *b(s = delete)*. For example, for the belief state *b = (1, 0)*, the left end of the line segment indicates *b(s = save) = 1* and *b(s = delete) = 0*. The intermediate points represent varying degrees of certainty in the user's goal.
The machine has three available actions: it can *ask* what the user wishes to do in order to infer his or her current goal, or it can *doSave* or *doDelete* and move to the next message. If the user says *save*, then an error may occur with probability 0.2, whereas if the user says *delete*, an error may occur with a probability 0.3.
The machine receives a large positive reward (+5) for getting the user's goal correct, a very large negative reward (-20) for taking the action *doDelete* when the user wanted *save*, and a smaller but still significant negative reward (-10) for taking the action *doSave* when the user wanted *delete*. There is also a small negative reward for taking the *ask* action (-1). The discount factor is set to 0.95 for this example.
Let's define the POMDP.

In [ ]:
```
# transition function P(s'|s,a)
t_prob = [[[0.65, 0.35], [0.65, 0.35]], [[0.65, 0.35], [0.65, 0.35]], [[1.0,
# evidence function P(e|s)
e_prob = [[[0.5, 0.5], [0.5, 0.5]], [[0.5, 0.5], [0.5, 0.5]], [[0.8, 0.2], [
# reward function
rewards = [[5, -10], [-20, 5], [-1, -1]]

gamma = 0.95
actions = ('0', '1', '2')
states = ('0', '1')

pomdp = POMDP(actions, t_prob, e_prob, rewards, states, gamma)
```

We have defined the `POMDP` object. Let's run `pomdp_value_iteration` to find the utility function.

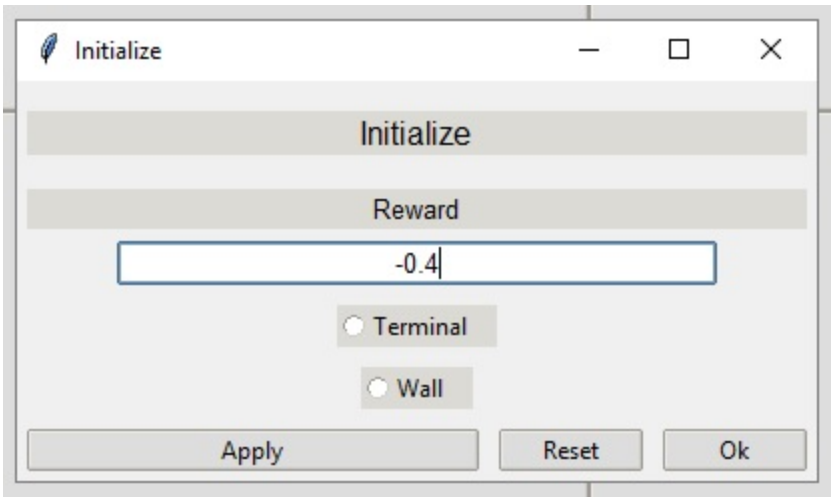
In [ ]:
```
utility = pomdp_value_iteration(pomdp, epsilon=0.1)
```

In [ ]:
```
%matplotlib inline
plot_pomdp_utility(utility)
```

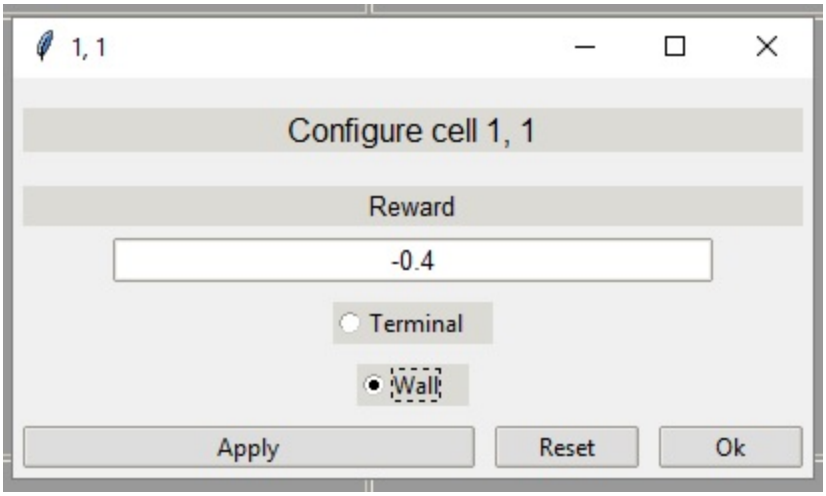
Appendix

Surprisingly, it turns out that there are six other optimal policies for various ranges of R(s). You can try to find them out for yourself. See **Exercise 17.5**. To help you with this, we have a GridMDP editor in `grid_mdp.py` in the GUI folder. Here's a brief tutorial about how to use it
Let us use it to solve `Case 2` above

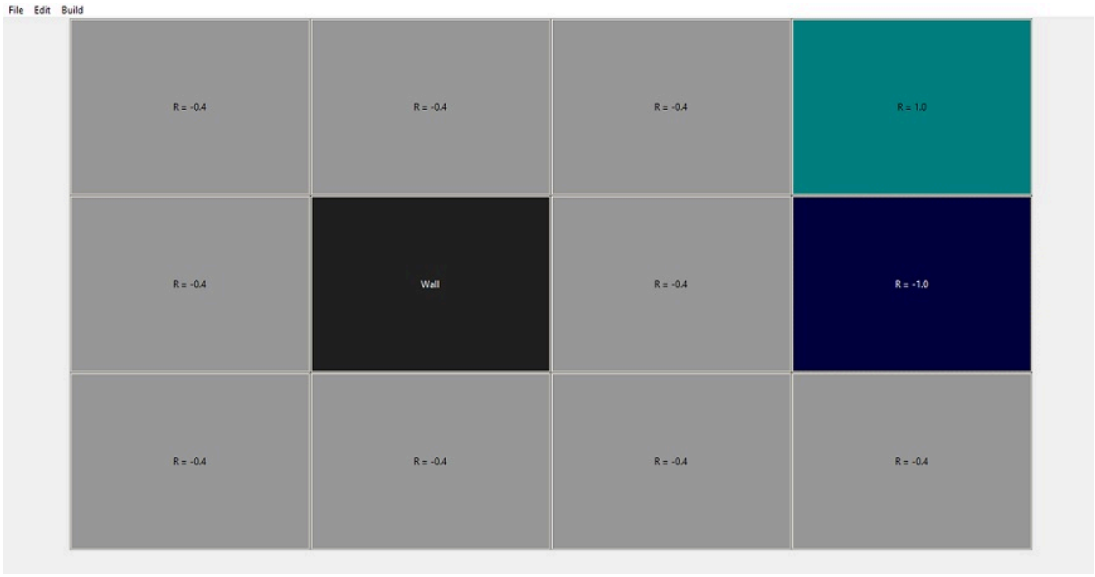
1. Run `python gui/grid_mdp.py` from the master directory.
2. Enter the dimensions of the grid (3 x 4 in this case), and click on `'Build a GridMDP'`
3. Click on `Initialize` in the `Edit` menu.
4. Set the reward as -0.4 and click `Apply`. Exit the dialog.



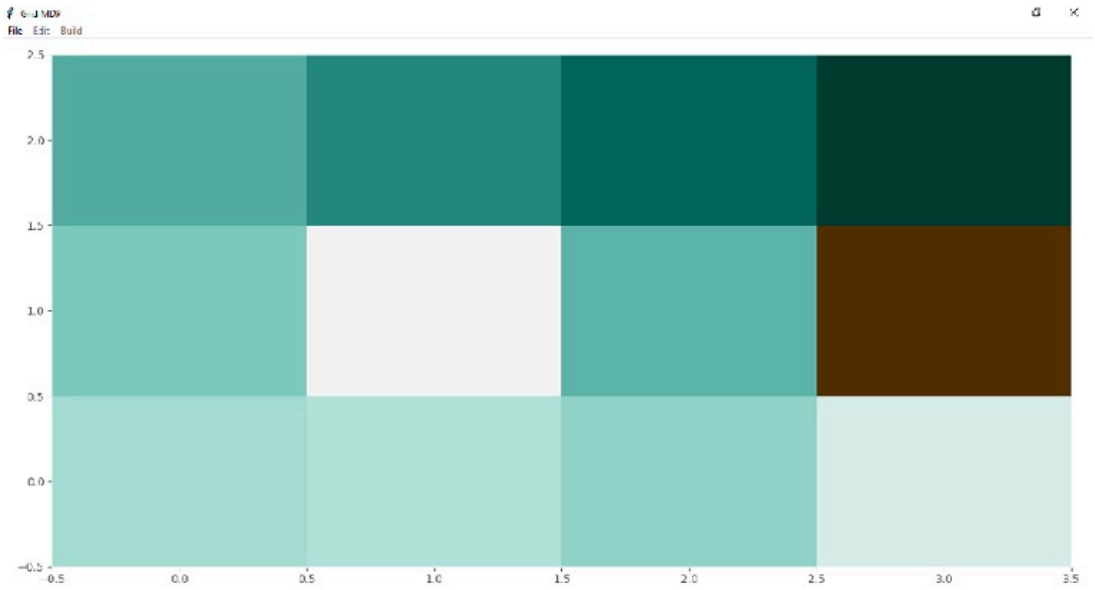
5. Select cell (1, 1) and check the `Wall` radio button. `Apply` and exit the dialog.
6. Check the appearance.



7. Select cells (4, 1) and (4, 2) and check the `Terminal` radio button for both. Set the rewards appropriately and click on `Apply`. Exit the dialog. Your window should look something like this.



8. You are all set up now. Click on `Build and Run` in the `Build` menu and watch the heatmap calculate the utility function.



Green shades indicate positive utilities and brown shades indicate negative utilities. The values of the utility function and arrow diagram will pop up in separate dialogs after the algorithm converges.

In [1]:
```
# Download your notebook, and upload it as mdp.pdf
!jupyter nbconvert --to webpdf mdp.ipynb
```

```
[NbConvertApp] Converting notebook mdp.ipynb to webpdf
[NbConvertApp] WARNING | Alternative text is missing on 1 image(s).
[NbConvertApp] Building PDF
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 262242 bytes to mdp.pdf
```

In [ ]:
```
```